

Phase 1 Report

Plant Disease Detection - Image Classification project

Farzaneh Khalili, Hasti Javazadeh, Atefeh Peimani

Overall summary

In phase 1 our goal was to prepare the groundwork for an image classification pipeline, dataset preparation, splitting, and augmentation. And also to design and implement a complete feature-extraction pipeline that prepares deep feature vectors from three CNN architectures: VGG16, ResNet50, and InceptionV3. These feature vectors will later be used to models such as SVM and KNN.

Dataset

The data is originally from PlantVillage (folder color), each subfolder corresponds to a class. The dataset contains of 38 classes belong to 14 plants, in the paper we also have 14 plants. We build a DataFrame with filepaths and labels. We performed a stratified split using `train_test_split` with `test_size=0.2` and `random_state=42`. Which produces 80% train and 10% test and 10% validation. Random seeds ensure reproducibility for these splits. For augmentation we used `ImageDataGenerator` with scaling (1./255) and transformation: rotation up to 40 degree width/height shifts 0.2, shear 0.2, zoom 0.2, horizontal flips, and `fill_mode='nearest'`. For each class we generated augmented images via `flow_from_dataframe` and saved to another folder. I also copied the original training images into the same training folders to have them with augmented images to have all of them in one folder.

Class weights and imbalance handling

Because class frequencies are imbalanced, we computed class weights using `comput_class_weight('balanced', classes, y)` and created a dictionary for use in `model.fit()` to reduce bias toward majority classes.

Baseline model implementation

We implemented a simple CNN baseline with following layers: Conv2D(16) + MaxPool, , Conv2D(32) + MaxPool, Flatten, Dense(64), and final Dense with softmax for `num_classes`. The model was compiled with Adam (`lr=0.001`), categorical crossentropy loss, and accuracy metric. Training was run for a quick test with `epochs=3` to confirm the pipeline (Training with `epochs=3` is only for quick validation) we had accuracy of 74%. Model summary and basic evaluation on the test generator were printed.

Feature Extraction Pipeline

We began by organizing the dataset in a way that makes it possible to process it across all models. We generated two mappings: class-name to numerical label and numerical label to class-name. These mappings were saved as a JSON file so that all later phases of the project use the same class indexing.

Next, we implemented the preprocessing pipeline for image inputs. Which included resizing each image into the required input size for each CNN model, converting images to NumPy arrays, adding the batch dimension required by Keras models, and applying the appropriate preprocessing function such as `vgg16.preprocess_input`.

We then created three model-builder functions: one for VGG16, one for ResNet50, and one for InceptionV3. Each of these loads the ImageNet-pretrained network and removes the classification layer so that the model outputs only the feature vector (for example, the 4096-dim fully connected features from VGG16). These functions were written in a flexible way so that when training the final

models were finished, we can simply replace the ImageNet weights with the trained .h5 weights.

After that, we implemented the single-image extraction function. This allowed us to test the system on individual images and verify that the model, preprocessing, etc work correctly.

Originally, extracting features one image at a time is extremely slow and would take hours to complete. To address this, we implemented a fully optimized batched feature extraction function. Instead of processing images individually, this function loads a full batch of images into memory, preprocesses them together, and performs a single forward pass through the model. This approach reduces the total number of model calls and speeds up extraction. But on a local machine, it still was slow.

We moved the process to Google Colab, mounted Google Drive, and extracted features using Colab's GPU. We uploaded the dataset in a ZIP file, extracted it inside Colab, and ran the batched extractor. The feature files for VGG16, ResNet50, and InceptionV3 were successfully saved in .npy format for the train, validation, and test splits. During this step we also had to work around Google Drive storage limitations.

Since the final trained model weights aren't provided yet, we were not able to perform the final feature extraction using the trained models. Instead, in this phase we created the full pipeline, tested it on sample images, optimized its performance, validated it on Google Colab, and prepared everything so that once the trained weights are available, the entire process can be executed again quickly.

Overall, Phase 1 successfully produced a complete and ready-to-use feature extraction system, including dataset handling, preprocessing, model preparation, single-image and batched extraction, and partial extraction of all three CNN architectures. The SVM and KNN training steps will be completed in the next phase after the final feature vectors from the trained models become available.

Training & Model Management Preparation

Although the full model training and model-saving pipeline will be executed in Phase 2, in Phase 1 we prepared all the structural components required for reproducible training.

This included designing the configuration system for model runs, deciding the metadata format for storing hyperparameters, and planning the logging and saving procedure for the trained models.

Specifically

A folder structure for saving models, logs, and metadata (models/, logs/, metadata/)

A JSON-based metadata template that stores model architecture, training parameters, dataset version, augmentation settings, class weights, and runtime information

A logging plan for accuracy, loss, and learning curves (TensorBoard + CSV fallback)

A reproducibility plan including fixed random seeds, environment documentation, and consistent preprocessing configurations

Together, these preparations ensure that Phase 2 model training can be performed consistently, with **complete traceability**.

Literature Insight for TO-DO work: Incorporating Uncertainty Estimation Using Bayesian Dropout (*previously discussed during a meeting*)

During Phase 1, we explored how uncertainty estimation can be incorporated into our plant disease detection pipeline, based on the frequently cited work “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning” by Gal & Ghahramani (2016). Although our current models (VGG16, ResNet50, InceptionV3 feature extractors + classical classifiers) do not yet implement

uncertainty estimation, understanding this framework will guide the design of Phase 2 and 3.

What we found out from the paper:

The paper shows that dropout is not only a regularization technique, but also a form of approximate Bayesian inference.

By keeping dropout active at test time and performing multiple stochastic forward passes (known as Monte Carlo Dropout), the model effectively samples from a posterior distribution over its weights.

This allows us to estimate two important quantities:

Predictive mean (average class prediction)

Predictive uncertainty (variance across forward passes)

This is especially relevant for plant disease diagnosis, where certain classes are visually similar, and uncertainty can help **avoid overconfident** wrong predictions.

How This Can Be Added to Our Project:

In the CNN Feature Extractors: Instead of using deterministic pretrained models, we can enable dropout inside the convolutional layers and keep it on during inference. Running the image through the extractor K times will produce K feature vectors, from which we can compute feature variance.

Although uncertainty methods were not implemented in Phase 1, we plan to explore the following in Phase 3 (if time allows):

Apply MC dropout inside the feature extractors to obtain uncertainty-aware feature vectors.

Compare SVM/KNN accuracy with and without uncertainty information.

Exploring explainability:

During Phase 1, and following the course professor's recommendation, we explored the topic of explainable artificial intelligence (XAI) and how explainability techniques can be incorporated into our plant disease detection pipeline.

Explainability is important for providing insights into why a model makes a certain prediction, especially in high-stakes or visually complex tasks like plant disease classification.

Why Explainability Matters in Our Context

Plant disease datasets often include subtle visual differences between classes, and the classification decisions can be influenced by small color or texture patterns. Explainability helps verifying that the model relies on meaningful features (disease spots, leaf edges) rather than irrelevant background noise and allow users (e.g., plant experts, students) to understand what parts of the leaf image contributed to the decision.

It can also help in spotting mislabeled images or biases if the model focuses on the wrong regions.

We plan to integrate explainability at two points in the pipeline:

Visualization of CNN Feature Extractors: Use Grad-CAM or Integrated Gradients on VGG16/ResNet50/InceptionV3. Then compare how different architectures highlight disease patterns

Explaining Final Predictions: If we build a full CNN classifier later, apply Grad-CAM directly on the final model

These insights will help us confirm whether the model is learning biologically meaningful features and will allow better debugging if certain classes are confusing.